

Xác định vị trí tâm vật thể trong miền tọa độ Descartes qua tọa độ miền ảnh pixels.

Nguyễn Huyền Trang

- 20020727 - K65R -

Đại học Công Nghệ, Đại học Quốc gia Hà Nội

htrang.cyb@gmail.com

Tóm tắt— Trọng tâm của một vật là điểm trung vị của một vật, khối lượng của vật phân bố đều xung quanh điểm này. Đây là nền tảng cơ bản của tất cả các bài toán tính toán cơ học thiết yếu, việc xác định trọng tâm là một bài toán quan trọng cần cân nhắc trong quá trình xây dựng và điều khiển một hệ thống cơ khí, dây chuyền sản xuất, hệ thống tay máy, robot... Trong báo cáo dưới đây trình bày phương pháp xác định trọng tâm của một vật thể bất kì thông qua xử lý một ảnh chụp đầu vào từ camera của Raspberry Pi. Chương trình áp dụng một số phương pháp xử lý và tiền xử lý ảnh đầu vào để nhận dạng vật thể và xác định tọa độ của điểm này chuyển từ hệ tọa độ ảnh sang hệ tọa độ không gian.

Từ khóa— trọng tâm vật, camera calibration, python-based, xử lý ảnh, OpenCV.

I. ĐẶT VẤN ĐỀ

Trọng tâm của một vật rắn là điểm tập trung khối lượng vật đó, là vị trí trung bình tới tất cả các phần vật chất tạo nên vật đó. Tâm của vật thể là một đặc trưng hợp lệ để biểu diễn một vật và là cơ sở để phân tích vật đó. Đây là một điểm tham chiếu quan trọng trong tính toán cơ học, và do đó, là một đặc trưng quan trọng cần xét đến trong các lĩnh vực kỹ thuật khác nhau, đặc biệt trong các bài toán thiết kế, điều khiển và kiểm thử. Trong ngành robot, trọng tâm cần được xác định trong bài toán thiết kế, chế tạo mô hình robot bởi nó là yếu tố ảnh hưởng trực tiếp đến khả năng thăng bằng và quá trình vận hành của robot; tâm cũng là đặc trưng trong bài toán xác định vật thể và là tọa độ đầu vào cho robot cần di chuyển đến khi thực hiện thao tác với vật. Để xác định tâm của vật so với hệ tọa độ gốc bằng đo đạc là điều không có tính áp dụng cao trong những vấn đề điều khiển robot hoạt động liên tục không nghỉ với hình dáng vật thể khác nhau sau mỗi lần thao tác. Bài báo cáo đưa ra đề xuất về phương pháp thực hiện tự động hóa trong xác định tọa độ tâm vật thể trong miền không gian thực từ ảnh chụp với hệ camera chụp từ trên xuống, chương trình sẽ xử lý độ lệch méo của ảnh, nhận dạng vật và xác định tâm của vật trong miền tọa độ ảnh pixels và miền không gian ba chiều Descartes.

II. CƠ SỞ LÝ THUYẾT

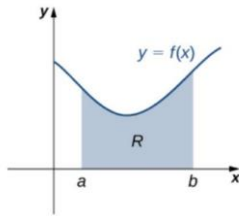
A. Tâm vật rắn.

Khi xét với vật rắn, cách xác định tâm của vật thay đổi tùy thuộc theo hình dáng của vật thể khi tính toán, độ phức tạp của phép tính phụ thuộc vào biên dạng bên ngoài của vật có đồng nhất hay không. Đối với hình khối cơ bản, tâm của vật nằm trên điểm giao nhau các trục đối xứng của chúng (hình 1).

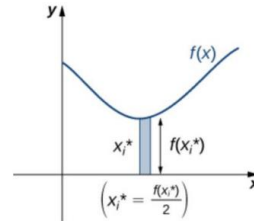


Hình 1. Biểu diễn tâm của một số hình khối cơ bản.

Tuy nhiên, với hầu hết mọi vật thể trong thực tế, với biên dạng và hình dáng ngẫu nhiên, việc xác định tâm của vật không thể xác định chính xác được bằng phương pháp đo đạc hay tính toán phổ thông. Trong trường hợp này, cách thực hiện xác định tâm của vật bằng tính tích phân khi chia biên dạng vật thành rất nhiều mảnh nhỏ gần giống hình cơ bản để có thể xác định được tâm của chúng, với số lượng tiến đến vô cực. Hình 2 và 3 biểu diễn ví dụ xấp xỉ xác định tâm một vật thể bất kỳ với biên trên xác định bởi hàm $y = f(x)$. Tại những vùng được sấp xỉ, tâm của vật có thể được xác định là: $\left(x_i, \frac{1}{2} f(x_i)\right)$.



Hình 2. Xác định tâm vùng R với giải tích.

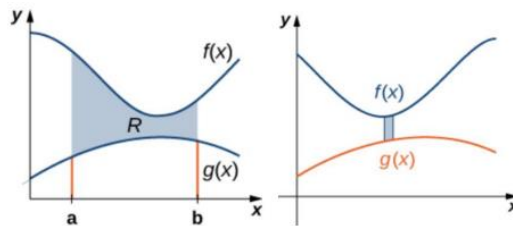


Hình 3. Sấp xỉ vùng R thành những hình khối nhỏ.

Tổng quát hóa như hình 4, khi biên trên và biên dưới của hình khối xác định bởi hàm $f(x)$ và $g(x)$, tâm của toàn thể vật được tính bằng:

$$\bar{x} = \frac{1}{A} \int_a^b x[f(x) - g(x)]dx$$

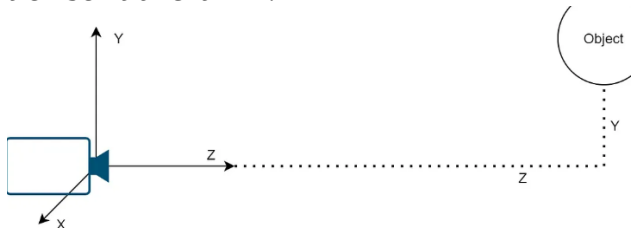
$$\bar{y} = \frac{1}{2A} \int_a^b \{[f(x)]^2 - [g(x)]^2\}dx \quad (1)$$



Hình 4. Xác định tâm vật với biên của vật xác định bởi hàm số.

B. Hệ tọa độ.

Hệ tọa độ camera là hệ tọa độ ba chiều Descartes với gốc tọa độ đặt ở tiêu cự của camera, trục X là trục thẳng đứng vuông góc xuống chiếu từ camera, mặt phẳng XY vuông góc với trục Z . Hệ tọa độ pixel được biểu diễn khi chia ảnh thành một lưới các ô vuông theo kích thước ảnh, ta cần xác định vị trí tâm vật thể nằm ở ô pixel thứ mấy tính từ ô (0,0) tại góc trên bên trái của ảnh.



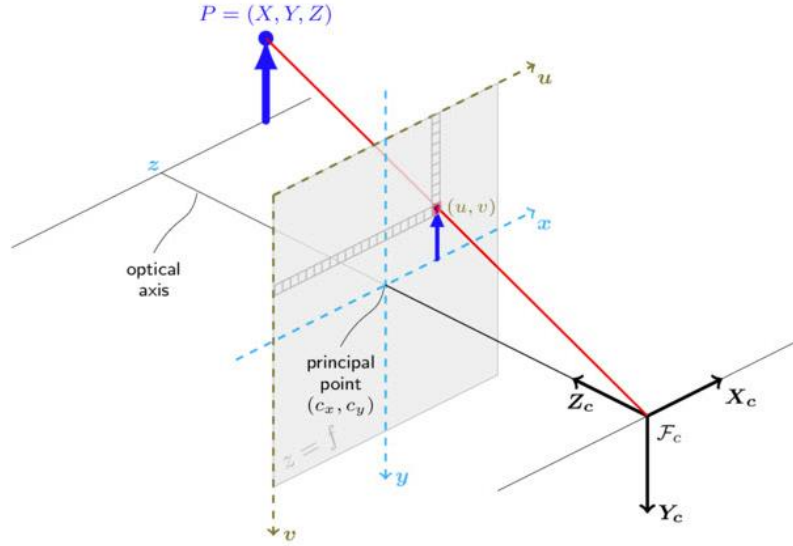
Hình 5. Biểu diễn hệ tọa độ của camera.

(0, 0)	(0, 1)	(0, 2)	(0, ...)	
(1, 0)	(1, 1)			
(2, 0)				
(..., 0)				

Hình 6. Biểu diễn hệ tọa độ ảnh pixels.

Mô hình mối tương quan giữa hệ tọa độ ảnh và hệ tọa độ gốc được biểu diễn như sau:

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_1 \\ r_{21} & r_{22} & r_{23} & t_2 \\ r_{31} & r_{32} & r_{33} & t_3 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \quad (1)$$



Hình 7. Mối quan hệ giữa miền tọa độ ảnh và miền tọa độ gốc
Trong đó: (X, Y, Z) là tọa độ điểm trong không gian ba chiều.

(u, v) là tọa độ trong không gian ảnh

Thực hiện Camera calibration ta tìm được các ma trận chuyển vị, ma trận xoay, ma trận đặc tính của camera; thực hiện nhận diện vật, xác định tâm ta có được tọa độ pixel, từ các tham số trên ta tìm được tọa độ của tâm vật trong không gian ba chiều. Các thông số của ma trận camera sẽ được trình bày rõ hơn ở phần sau.

C. Camera Calibration

Camera calibration là quá trình xác định các yếu tố nội tại và ngoại tại ảnh hưởng lên ảnh chụp của camera. Các thành phần ngoại tại biểu diễn mối quan hệ giữa hệ tọa độ 3D trong thế giới thực với vị trí đặt camera. Các thành phần nội tại thể hiện mối quan hệ giữa hệ tọa độ Descartes đã đặt với hệ tọa độ pixel trong ảnh mà camera chụp lại.

Mục đích của bài toán chỉ tập trung vào thành phần nội tại ảnh hưởng đến ảnh của camera, phương trình dưới đây biểu diễn mô hình toán học của các thành phần nội tại của camera.

$$\begin{bmatrix} k * x \\ k * y \\ k \end{bmatrix} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} \quad (2)$$

Trong đó, ma trận đầu tiên biểu diễn hệ tọa độ pixel, ma trận thứ 2 biểu diễn các thành phần nội tại camera, ma trận thứ 3 biểu diễn tọa độ. Giá trị X, Y, Z là tọa độ của một điểm trong thế giới thực. f_x, f_y là khoảng tập trung của camera theo trục X và Y . Với mô hình pinhole camera, ánh sáng lọt qua cửa chụp là một lỗ tròn nhỏ để được thu lại thành ảnh trên film, do đó f_x, f_y bằng nhau và bằng đơn vị khoảng cách. Tuy nhiên với camera điện tử như

của Pi, các cảm biến thu ảnh và ống kính của camera có độ méo ảnh nhất định, thông số f_x, f_y khác nhau và khác biệt giữa mỗi loại camera.

Công thức dưới đây thể hiện lý thuyết về ‘focus in pixels’ – khoảng tập trung trong miền pixels của một camera điện tử.

$$f_x = \frac{FW}{w} \quad (3)$$

Với f_x là tầm focus trong miền pixel của trục X , F là tiêu cự của camera (mm), W là độ rộng của cảm biến trong camera (mm), w là độ rộng ảnh trong miền pixel.

D. Tinh chỉnh độ biến dạng camera.

Tham khảo của paper [1] của tác giả Zhen Liu và cộng sự thực hiện căn chỉnh ảnh camera rẻ với chất lượng thấp để loại bỏ độ méo của ảnh, tăng độ phân giải, lọc nhiễu. Áp dụng vào bài toán tìm 4 vị trí góc bàn cờ bao quanh môi trường làm việc, sử dụng hàm căn chỉnh camera `cv.calibrateCamera()`, `cv.undistort()`, `remapping` để loại bỏ độ méo trên ảnh và `warp.perspective()` để đưa ảnh về biên dạng đúng khi chụp từ trên xuống.

Độ méo xuyên tâm – radial distortion của ảnh xảy ra khi chụp đường thẳng nhưng thu về ảnh bị biến thành đường cong, độ méo xuyên tâm càng lớn khi vật thể xếp càng xa khỏi trung tâm bức ảnh nơi đặt camera chiếu xuống. Công thức biểu diễn độ méo xuyên tâm:

$$\begin{aligned} x_{distorted} &= x + (2p_1xy + p_2(r^2 + 2x^2)) \\ y_{distorted} &= y + (p_1(r^2 + 2y^2) + 2p_2xy) \end{aligned} \quad (4)$$

Các thành phần biểu diễn độ méo của ảnh được thể hiện dưới dạng vector:

$$Distortion_coefficients = (k_1 \quad k_2 \quad p_1 \quad p_2 \quad k_3) \quad (5)$$

III. PHƯƠNG PHÁP THỰC HIỆN

A. Cài đặt camera.

Tải thư viện chứa môi trường cần thiết để kết nối PiCamera với Python.

```
PS D:\Work\N4\IP\49_NguyenHuyenTrang> sudo apt-get install python-picamera python3-picamera python-rpi.gpio
PS D:\Work\N4\IP\49_NguyenHuyenTrang> pip install picamera
Defaulting to user installation because normal site-packages is not writeable
Collecting picamera
  Downloading picamera-1.13.tar.gz (143 kB)
    143.8/143.8 kB 947.5 kB/s eta 0:00:00
Installing build dependencies ... /
```

Kích hoạt camera của Raspberry Pi.

- Kết nối cáp USB.
- Đăng nhập Pi với username `pi` và mật khẩu.
- Nhập `sudo raspi-config` vào command prompt
- Mở menu, tìm và chọn Enable Camera
- Chọn Enable -> Finish -> Yes (reboot).

```
from picamera.array import PiRGBArray //thêm thư viện PiCamera
from picamera import PiCamera        //làm việc với Python
```

B. Camera Calibration.

- ▼ camera_data
 - ≡ cam_mtx.npy
 - ≡ dist.npy
 - ≡ newcam_mtx.npy
 - ≡ roi.npy

C. Xác định tâm vật thể.

Ta thực hiện xác định các vật thể thông qua xác định contour đường viền ngoài xung quanh vật. Chương trình nhận đầu vào là ảnh, thực hiện xác định cạnh biên của vật và vẽ contour xung quanh vật. Thực hiện xác định tâm của nhiều vật trên cùng một khung hình. Vì xác định tâm lấy đường contour viền ngoài hình chữ nhật nên cần xác định đường contour hợp lệ cho chương trình ở các bước sau.

```

47 def detectObjects(self, image, bg_img, externalContours=True):
48     img=image.copy()
49     background_img=bg_img.copy()
50     # Process Image Difference
51     diff=self.calculateDifference_Otsu(img,background_img)
52
53     # Xác định contour của vật
54     # dùng RETR_EXTERNAL chỉ xác định viền ngoài, dùng RETR_TREE để xác định tất cả các contour
55     if externalContours==True:
56         contours_detected, hierarchy = cv2.findContours(diff, cv2.RETR_EXTERNAL,
57                                                         cv2.CHAIN_APPROX_SIMPLE)
58     else:
59         contours_detected, hierarchy = cv2.findContours(diff, cv2.RETR_TREE,
60                                                         cv2.CHAIN_APPROX_SIMPLE)
61
62     # tính các tham số cần thiết
63     height, width, channels = img.shape
64
65     # xác định tất cả các contour hợp lệ
66     contours_validindex= self.identify_validcontours(contours_detected,height,width)
67     obj_count=len(contours_validindex)
68     self.printStatus("valid contours "+ str(obj_count))
69
70     return obj_count, contours_detected, contours_validindex
    
```

Sau khi xác định được số contour trên hình, ta xác định số lượng các vật thể vào thêm chúng vào một mảng lưu.

```

38 def run_detection(self,img,bg,testRun=False):
39
40     obj_count, contours_detected, contours_validindex=self.detectObjects(img,bg)
41
42     obj_count, detected_points, img_output=self.detectionOutput(img, obj_count,
43     contours_validindex,
44     contours_detected)
45     return obj_count, detected_points, img_output
    
```

Duyệt tất cả các contour hợp lệ trong khung hình, vẽ hình chữ nhật bao quanh vật thể và xác định điểm tâm của vật là điểm giao của hai đường chéo.

```

78         if (len(validcontours)>0):
79             for i in range(0,len(validcontours)):
80                 cnt=diff_contours[validcontours[i]]
81
82                 # ve duong vien bao quanh vat hình chu nhật
83                 x,y,w,h=cv2.boundingRect(cnt)
84
85                 # Lay tam vat the
86                 M=cv2.moments(cnt)
87                 cx=int(M['m10']/M['m00'])
88                 cy=int(M['m01']/M['m00'])
89
90                 self.printStatus("point number "+str(i))
91                 self.printStatus(str(cx)+", "+str(y))
92                 self.printStatus("x: "+str(x)+" y: "+str(y)+" w: "+str(w)+" h: "+str(h))

```

Sau khi xác định được tâm vật, ta hiển thị tọa độ trong miền không gian ảnh của vật và lưu các thông tin về tọa độ ảnh và tọa độ gốc của tâm vật vào danh sách đợi.

```

#image, text, font, bottomLeftconrner, fontscale, fontcolor, Linetype
cv2.putText(img_output, "Point "+str(i), (x-w,y+h), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255,0,0), 1)
cv2.putText(img_output, "cx,cy: "+str(self.truncate(cx,2))+", "+str(self.truncate(cy,2)), (x-w,y+h+9),

points=[x,y,w,h,cx,cy]
detected_points.append(points)

```

D. Căn chỉnh tọa độ điểm theo góc chụp.

Từ các ma trận thông số camera đã xác định, ta điều chỉnh ma trận đầu vào cho camera để căn lại cách chụp ảnh, chỉnh độ méo vật xuống.

```

6     cameraXYZ=camera_realworldxyz.camera_realtimeXYZ()
16    #Load camera calibration
17    savedir="camera_data/"
18    cam_mtx=np.load(savedir+'cam_mtx.npy')
19    dist=np.load(savedir+'dist.npy')
20    newcam_mtx=np.load(savedir+'newcam_mtx.npy')
21    roi=np.load(savedir+'roi.npy')
22
23    #Load center points from New Camera matrix
24    cx=newcam_mtx[0,2]
25    cy=newcam_mtx[1,2]
26    fx=newcam_mtx[0,0]
27    print("cx: "+str(cx)+" cy "+str(cy)+" fx "+str(fx))

```

trong đó, hàm camera_realtimeXYZ() thực hiện khởi tạo lại các thông số camera như sau:

```

30    self.cam_mtx=np.load(savedir+'cam_mtx.npy')
31    self.dist=np.load(savedir+'dist.npy')
32    self.newcam_mtx=np.load(savedir+'newcam_mtx.npy')
33    self.roi=np.load(savedir+'roi.npy')
34    self.rvec1=np.load(savedir+'rvec1.npy')
35    self.tvec1=np.load(savedir+'tvec1.npy')
36    self.R_mtx=np.load(savedir+'R_mtx.npy')
37    self.Rt=np.load(savedir+'Rt.npy')
38    self.P_mtx=np.load(savedir+'P_mtx.npy')
39
40    s_arr=np.load(savedir+'s_arr.npy')
41    self.scalingfactor=s_arr[0]
42
43    self.inverse_newcam_mtx = np.linalg.inv(self.newcam_mtx)
44    self.inverse_R_mtx = np.linalg.inv(self.R_mtx)

```

Xác định tọa độ không gian ảnh pixels:

```
143 XYZ1=np.array([[worldPoints[i,0],worldPoints[i,1],worldPoints[i,2],1]], dtype=np.float32)
144 XYZ1=XYZ1.T
145 print("{-- XYZ1")
146 print(XYZ1)
147 suv1=P_mtx.dot(XYZ1)
148 print("//-- suv1")
149 print(suv1)
150 s=suv1[2,0]
151 uv1=suv1/s
152 print(">==> uv1 - Image Points")
153 print(uv1)
154 print(">==> s - Scaling Factor")
155 print(s)
156 s_arr=np.array([s/total_points_used+s_arr[0]], dtype=np.float32)
157 s_describe[i]=s
158 if writeValues==True: np.save(savedir+'s_arr.npy', s_arr)
```

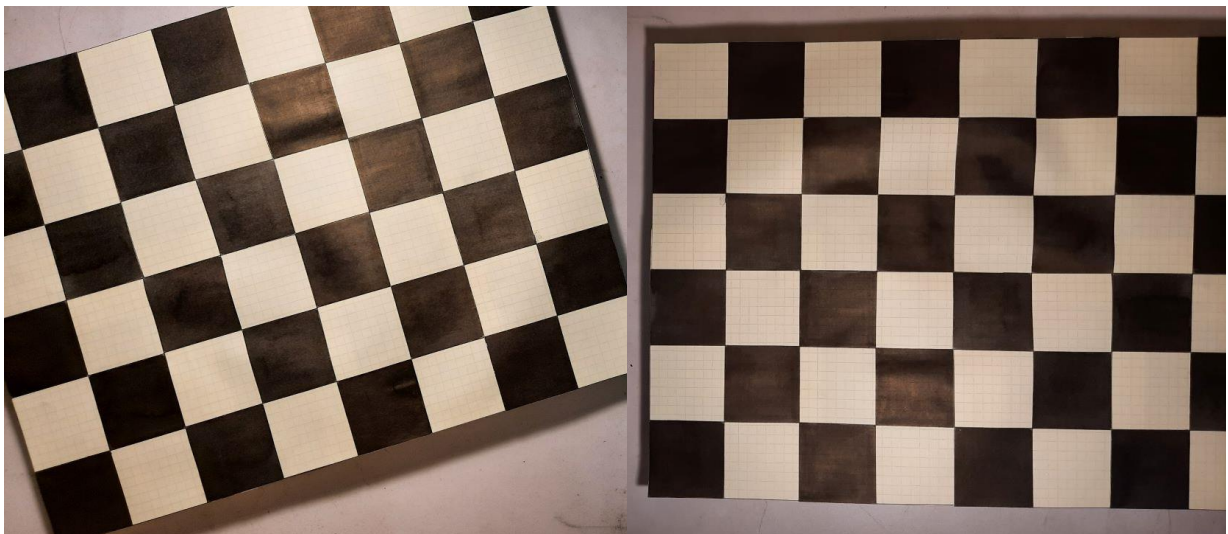
Tính tọa độ trong không gian ba chiều (X,Y,Z) từ tọa độ điểm ảnh.

```
162 uv_1=np.array([[imagePoints[i,0],imagePoints[i,1],1]], dtype=np.float32)
163 uv_1=uv_1.T
164 print(">==> uv1")
165 print(uv_1)
166 suv_1=s*uv_1
171 xyz_c=inverse_newcam_mtx.dot(suv_1)
172 xyz_c=xyz_c+tvec1
173 print("xyz_c")
174 inverse_R_mtx = np.linalg.inv(R_mtx)
175 XYZ=inverse_R_mtx.dot(xyz_c)
179 if calculatefromCam==True:
180     cXYZ=cameraXYZ.calculate_XYZ(imagePoints[i,0],imagePoints[i,1])
181     print("camXYZ")
182     print(cXYZ)
```

IV. KẾT QUẢ VÀ ĐÁNH GIÁ

A. Camera Calibration.

Xử lý dữ liệu ảnh đầu vào cho quá trình căn chỉnh camera được lưu trong thư mục /calibration_images gồm 56 ảnh kích thước 1280x720. Hình biểu diễn một ảnh đầu vào trong tập mẫu.



Hình 8. Ảnh đầu vào quá trình Calibrate.

Ta tính được các ma trận và vector thông số camera như sau:

Ma trận camera:

$$Camera_matrix = \begin{bmatrix} 9.95408646e+02 & 0.00000000e+00 & 6.37874652e+02 \\ 0.00000000e+00 & 1.00004273e+03 & 3.55610380e+02 \\ 0.00000000e+00 & 0.00000000e+00 & 1.00000000e+00 \end{bmatrix} \quad (6)$$

Hệ số biến dạng:

$$Distortion_coefficients = (0.15247988 \quad -0.46548877 \quad -0.01025309 \quad 0.006095 \quad 1) \quad (7)$$

Vector r

$$r = \begin{bmatrix} -0.0200916 \\ -0.00781944 \\ -1.56067222 \end{bmatrix} \quad (8)$$

Vector t

$$t = \begin{bmatrix} -5.98837843 \\ 10.26037967 \\ 48.46336221 \end{bmatrix} \quad (9)$$

Kích thước ảnh: 1280 x 720. Vùng tập trung – Region of Interest: (9, 7, 1262, 704)

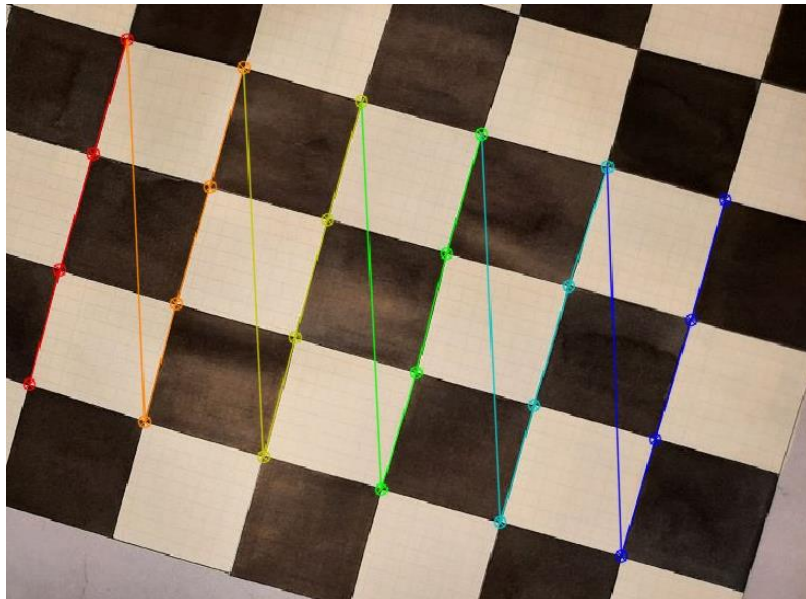
Mã trận camera mới:

$$new_camera_matrix = \begin{bmatrix} 1.00319562e+03 & 0.00000000e+00 & 6.45906451e+02 \\ 0.00000000e+00 & 1.00362935e+03 & 3.48827693e+02 \\ 0.00000000e+00 & 0.00000000e+00 & 1.00000000e+00 \end{bmatrix} \quad (6)$$

Mã trận camera nghịch đảo:

$$inv_new_cmatrix = \begin{bmatrix} 9.96814563e-04 & 0.00000000e+00 & -6.43848957e-01 \\ 0.00000000e+00 & 9.96383778e-04 & -3.47566255e-01 \\ 0.00000000e+00 & 0.00000000e+00 & 1.00000000e+00 \end{bmatrix} \quad (7)$$

Kết quả hiển thị được lưu trong thư mục /calib_results:

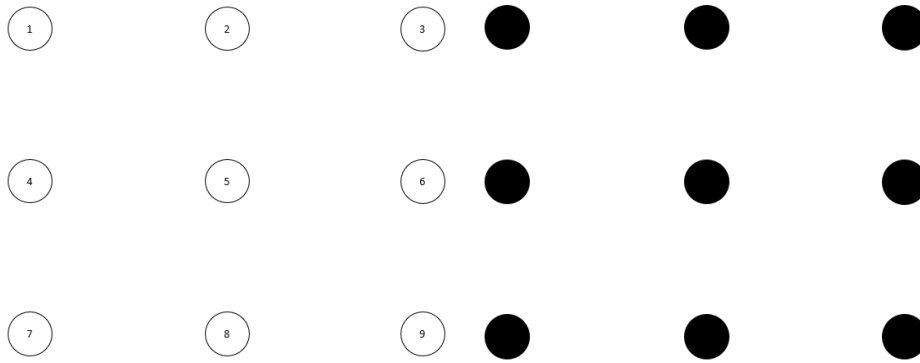


Hình 9. Kết quả hiển thị xác định điểm trên bàn cờ.

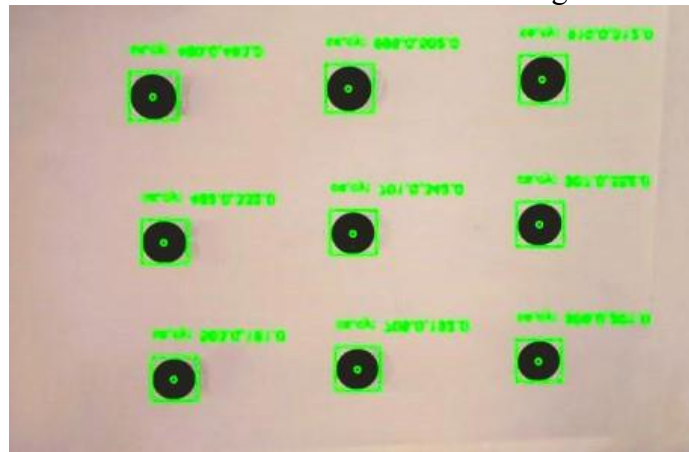
B. Xác định vị trí vật.

Sử dụng ảnh đầu vào 9 điểm để tính độ chính xác của chương trình.

Đầu vào:



Hình 10. Ảnh mẫu đầu vào cho chương trình.



Hình 11. Kết quả xác định tâm và tọa độ của các vật.

Đặt vật thể trên một mặt phẳng nền kẻ ô vuông bàn cờ, kích thước mỗi ô của bàn cờ là 5x5cm. Trục tọa độ X, Y của hệ được gắn ở góc trái bên trên của bàn cờ, trục Z hướng lên đảm bảo mọi tọa độ (x,y,z) xác định cho tâm vật thể đều mang giá trị dương.

Để xác định được vị trí của tâm vật thể trong không gian với 3 tọa độ theo trục XYZ, camera đặt chụp từ trên xuống cách mặt phẳng đặt vật 50cm. Tại vị trí pixel (645,368) ta xác định được là điểm chính giữa tính từ camera xét xuống, đánh dấu điểm sát nhất trên hình ta xác định được tọa độ (X_center, Y_center, Z_center) như dưới đây:

```
40 X_center=12.6
41 Y_center=16.7
42 Z_center=50
43 worldPoints=np.array([[X_center,Y_center,Z_center],
44                        [5.3,4.3,53.4],
45                        [18.1,4.3,52.2],
46                        [30.4,4.3,58.2],
47                        [5.3,12.9,50.3],
48                        [18.1,12.9,51.6],
49                        [30.4,8.12.9,56.9],
50                        [5.3,21.8,51.1],
51                        [18.1,21.8,50.5],
52                        [30.4,21.8,55.6]], dtype=np.float32)
```

Tọa độ tâm các vật theo miền pixel xác định được từ chương trình.

```
57 imagePoints=np.array([[cx,cy],
58                       [491,167],
59                       [712,188],
60                       [883,219],
61                       [506,324],
62                       [671,351],
63                       [892,368],
64                       [479,482],
65                       [681,504],
66                       [897,536]], dtype=np.float32)
```

Sử dụng hàm tính tọa độ tâm với những tọa độ ảnh ở trên ta tính được tọa độ mẫu và độ chính xác của các vị trí xác định được như sau:

```
Point 1
S: 52.56472   Mean: 51.98665   Error: 1.2755354
Point 2
S: 51.76454   Mean: 50.43412   Error: 1.7756113
Point 3
S: 55.64702   Mean: 57.16431   Error: -2.5316887
Point 4
S: 49.98925   Mean: 50.22967   Error: -1.2064443
Point 5
S: 52.01082   Mean: 51.71165   Error: 1.4931450
Point 6
S: 56.36489   Mean: 56.62718   Error: -1.3136274
Point 7
S: 52.93602   Mean: 51.98313   Error: 2.314169
Point 8
S: 49.63225   Mean: 50.22967   Error: -1.7974452
Point 9
S: 55.28931   Mean: 55.41458   Error: -1.0112782
```

Ta thấy tọa độ vị trí tâm vật thể đã xác định được gần đúng so với tọa độ thực. Độ sai số khoảng 2% tại các mẫu đo đặc thực tế. Tuy nhiên báo cáo vẫn đang gặp khó khăn khi xác định tâm của vật thể không phẳng, khi chiều cao của vật lớn hơn $\frac{1}{4}$ chiều dài hoặc chiều rộng của vật thì độ sai số không thấp ở mức 2% mà tăng lên 14.63% và có trường hợp không xác định được tâm của vật.

KẾT LUẬN

Trên ảnh đầu vào chụp từ camera Raspberry Pi, báo cáo đã thực hiện được quá trình căn chỉnh thông số của camera, xác định được các ma trận thông số nội tại của camera và các vector chuyển vị, vector biến dạng. Bài cũng đã xác định được các bước xác định biên cạnh, nhận dạng vật, xác định được tâm của vật đối với một số trường hợp cơ bản. Từ các thông số xác định trên, bài đã thực hiện được quá trình tìm tọa độ điểm trong miền pixel và chuyển tọa độ này về tọa độ trong không gian ba chiều Descartes với trục tọa độ gốc gắn ở tâm của camera.

TÀI LIỆU THAM KHẢO

- [1] Stewart, J. (2012). Essential Calculus: Early Transcendentals. Mason, Ohio: Cengage Learning. Young, H., Freedman, R (2012). University Physics with Modern Physics, 14e. Pearson Learning.
- [2] Demaagd K., Oliver A. (2012) Practical Computer Vision with SimpleCV. O'Reilly Media.
- [3] Mathematics LibreTexts. "Moments and centers of mass"
- [4] Horn, B. K. P. (1986); Robot vision. The MIT Press, Massachusetts.
- [5] Amina Gahramanova, Locating Centers of Mass with Image Processing, DOI:10.5038/2326-3652.10.1.4906
- [6] Tutorial OpenCV: Camera Calibration and 3D Reconstruction: Camera Calibration
- [7] Calculate X, Y, Z Real World Coordinates from Image Coordinates using OpenCV